MODEL CONTEXT PROTOCOL · DEMO

Help Desk MCP Server

How Claude looks up orders, issues refunds and opens tickets for a fictional store, with the rules enforced in code.

TypeScript · stdio · SQLite (node:sqlite) · Node 22.13+

HOW IT WORKS

One local process, started by the client



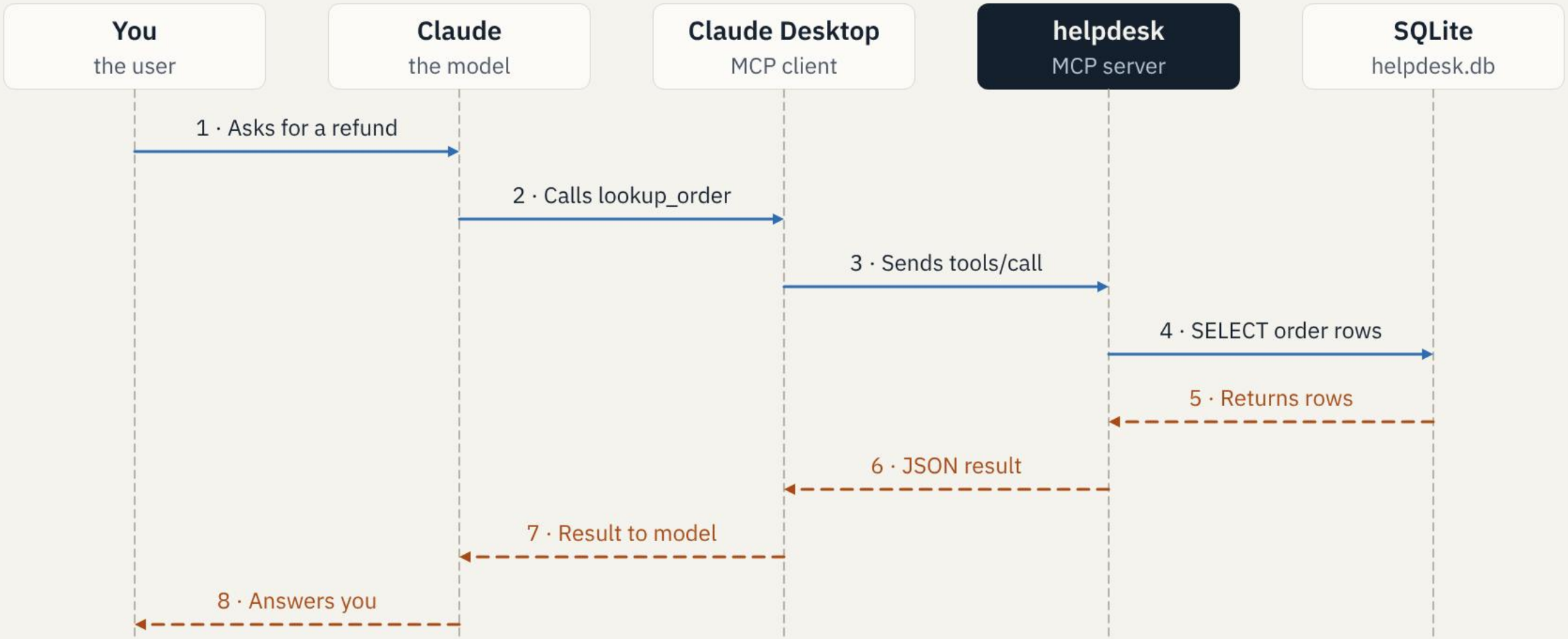
The client owns the process. It runs node dist/index.js and talks over stdin and stdout.

stdout is for the protocol only. Logs go to stderr, or they corrupt the stream.

One source for the rules. src/policy.ts feeds both the checks and the policy text.

HOW IT WORKS

From a question to SQLite and back



HOW IT WORKS

What each side actually sees

1 · Claude reads the tool list

```
{
  "name": "lookup_order",
  "description":
    "Get an order's status…",
  "inputSchema": {
    "properties": {
      "order_id": {
        "type": "string"
      }
    }
  }
}
```

2 · The client sends a request

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "lookup_order",
    "arguments": {
      "order_id": "ORD-5001"
    }
  }
}
```

3 · The server queries SQLite

```
SELECT * FROM orders
 WHERE id = ?;

SELECT sku, name, quantity,
  unit_price_cents,
  final_sale
  FROM order_items
 WHERE order_id = ?;

SELECT … FROM refunds
 WHERE order_id = ?;
```

Claude only sees tool names, schemas and JSON. The SQL and the database stay inside the server.

HOW IT WORKS

Seven things the server offers

Type	Name	What it does
Tool	find_customer	Search customers by name or email
Tool	lookup_order	Status, shipping, items, refunds and eligibility
Tool	issue_refund	Record a refund if the policy allows it
Tool	open_ticket	Create a ticket, optionally tied to an order
Tool	list_tickets	List tickets, most urgent first
Resource	helpdesk://policies/refunds	The refund policy, in Markdown
Prompt	draft_reply	Builds a reply from the ticket, order and policy

The model calls tools. The app reads resources. The user picks prompts.

HOW IT WORKS

Anatomy of a tool

```
server.registerTool(
  "lookup_order",
  {
    description: "Get an order's status, …",
    inputSchema: { order_id: z.string().min(1) },
    annotations: { readOnlyHint: true },
  },
  ({ order_id }) => {
    const order = describeOrder(order_id);
    return order
      ? ok(order)
      : fail("ORDER_NOT_FOUND", …);
  },
);
```

1. **Name and description**
   What the model reads to decide when to call the tool.
2. **Input schema**
   Zod validates the arguments before the handler runs.
3. **Annotations**
   Hints for the client, such as readOnlyHint and destructiveHint.
4. **Handler**
   Returns JSON through ok(), or an error code through fail().

DATA AND RULES

Every refund passes four checks

Check	Rejected with	Demo example
1. The order has been delivered	NOT_DELIVERED	Tom’s grinder is still in transit
2. It’s within 30 days of delivery	WINDOW_EXPIRED	Daniel’s backpack arrived 45 days ago
3. The amount fits what’s left, after final-sale items and earlier refunds	AMOUNT_EXCEEDS_REFUNDABLE	Sofia has $40 left: the scarf is excluded and $20 was refunded
4. Total refunds on the order stay at or under $250	SUPERVISOR_REQUIRED	Priya’s $412 desk needs a supervisor

The checks and the insert share one transaction. lookup_order reports the same verdict up front.

DATA AND RULES

Smaller safeguards that matter

stdout is for the protocol only

Logs go to stderr. A stray console.log would corrupt the message stream.

One transaction per refund

BEGIN IMMEDIATE stops two clients from refunding the same balance.

Customer text is data

draft_reply wraps the ticket in a record and tells the model not to follow it.

No splitting refunds

The $250 limit counts earlier refunds, so smaller pieces still need approval.

SET IT UP

Register it with Claude Desktop

1. **Build the server**

   npm install, then npm run build to create dist/index.js. npm test runs 11 end-to-end tests.

2. **Open the config file**

   Settings › Desktop app › Local MCP servers › Edit config opens claude_desktop_config.json.

3. **Add a helpdesk entry**

   Under mcpServers, set command to node and args to the absolute path of dist/index.js. On Windows, double every backslash.

4. **Restart Claude Desktop**

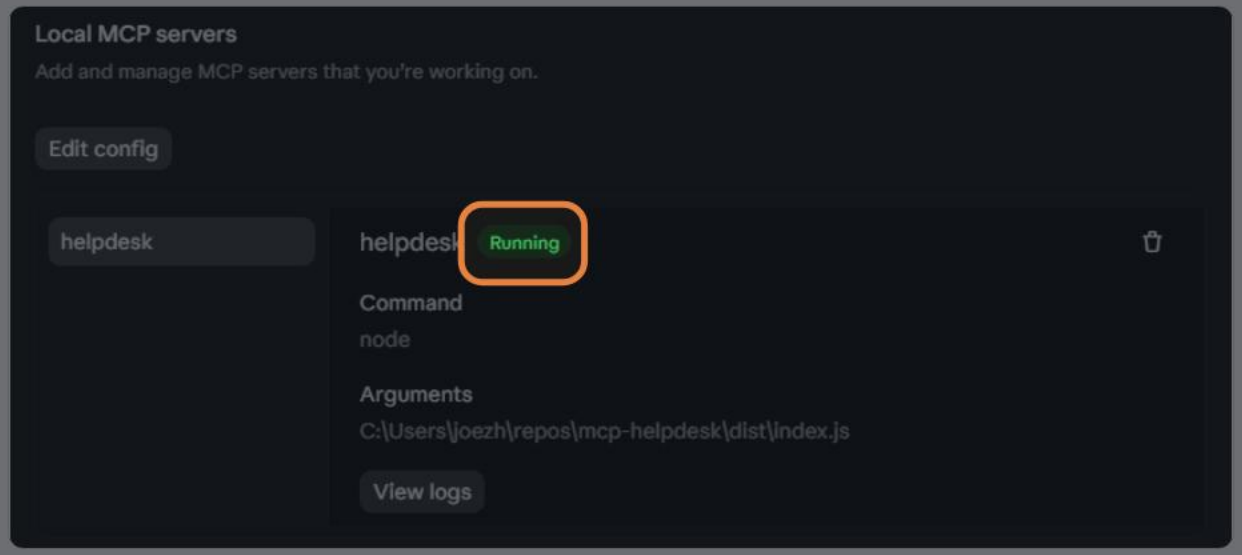
   Quit it completely, including from the system tray, then open it again.

```
{
  "mcpServers": {
    "helpdesk": {
      "command": "node",
      "args": [
        "C:\\Users\\joezh\\repos\\mcp-helpdesk\\dist\\index.js"
      ]
    }
  },
```

The entry in claude_desktop_config.json on Windows. Keep any other keys already in the file.

SET IT UP

Check that it’s running



Settings › Desktop app › Local MCP servers

Running

The server started and finished the MCP handshake.

Command and Arguments

Exactly what Desktop launches. Check the path if it fails.

View logs

Shows the server’s stderr. Start here when something’s wrong.

Changed the code?

Run npm run build, then restart Claude Desktop.

Running

DEMO 1 · PARTIAL REFUND

Refund only what was damaged

YOU ASK

“Handle TCK-1001 and refund what the customer is entitled to.”

Claude replies

Refunds $89.00 for the headphones and leaves the $14.50 cable alone, because only the headphones were damaged.

TOOL CALLS

1. `list_tickets()`
   → TCK-1001: headphones arrived damaged, the cable is fine
2. `lookup_order("ORD-5001")`
   → Eligible: $103.50 refundable, inside the 30-day window
3. `issue_refund("ORD-5001", 8900, "damaged")`
   → **RFD-2002**: $89.00 refunded, $14.50 left on the order

Claude chooses the amount. The server checks it against the policy before writing.

DEMO 2 · OVER THE LIMIT

A refund that needs a supervisor

YOU ASK

“Priya Raman wants a full refund on her desk.”

Claude replies

Explains that a supervisor must approve $412.00 and opens urgent ticket TCK-1006 with the details.

TOOL CALLS

1. `find_customer("Priya")`
   → CUS-1003, with order ORD-5003
2. `lookup_order("ORD-5003")`
   → Eligible: $412.00 refundable, supervisor needed above $250.00
3. `issue_refund("ORD-5003", 41200, "defective")`
   → **Rejected: SUPERVISOR_REQUIRED.** No refund was issued.
4. `open_ticket("CUS-1003", priority: "urgent")`
   → **TCK-1006** opened for supervisor approval

Claude may skip step 3 after reading the limit. If it tries anyway, the server refuses.

NEXT STEPS

Where to take it next

Add more tools

Each new capability is one registerTool call plus an end-to-end test.

Host it remotely

Swap stdio for Streamable HTTP and move the data to a hosted database, such as Cloudflare D1.

Add authentication

There’s none today. Require a token or OAuth before the server gets a public URL.